DISCRETE-TIME MODELLING OF A DC MOTOR

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....4

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Convert the continuous-time motor model into difference equations using the zero-order-hold and forward Euler methods.	☐
2	Run the discrete-time models alongside the DC motor in Simulink and measure their prediction error for several sample periods.	☐
3	Explain how the sample period and the discretisation method affect the accuracy and stability of a discrete-time model.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder and the measurement of motor speed in revolutions per minute (RPM). The set-up is not repeated here.

The first-order model of the motor, $G(s) = K/(\tau s + 1)$, identified in the practical exercise, Parameter Estimation of a DC Motor using Linear Least Squares, is used throughout. Use the values of K and τ identified for your own motor. Where example values are needed, this guide uses the A1.4 example results, $K = 28$ RPM/V and $\tau = 0.15$ s.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. The motor model is converted into two different difference equations, which run on the Arduino Uno alongside the real motor with the same input. Comparing their outputs with the measured speed shows how the sample period and the method affect accuracy. The exercise supports the ACE-CORE learning guide, Discrete-Time Modelling.

Derive the Difference Equations

When the input is held constant between samples, as it is by the PWM output, the exact discrete-time equivalent of $K/(\tau s + 1)$ is the zero-order-hold (ZOH) model:

$$\text{ZOH}: \quad y[k] = e^{-T_s/\tau}\, y[k-1] + K\left(1 - e^{-T_s/\tau}\right) u[k-1]$$

Approximating the derivative in the ODE $\tau\, dy/dt + y = K\, u$ with a forward difference gives the forward Euler model:

$$\text{Forward Euler}: \quad y[k] = (1 - T_s/\tau)\, y[k-1] + (KT_s/\tau)\, u[k-1]$$

The coefficient on $y[k-1]$ is the pole of each model in the z-plane. For the ZOH model it always lies between 0 and 1. For forward Euler it becomes negative when $T_s > \tau$, and leaves the unit circle, making the model unstable, when $T_s > 2\tau$.

The pole of each model is plotted against the sample period in Figure 1 for the example motor. The two agree closely when T_s is much smaller than τ, and diverge as it grows.

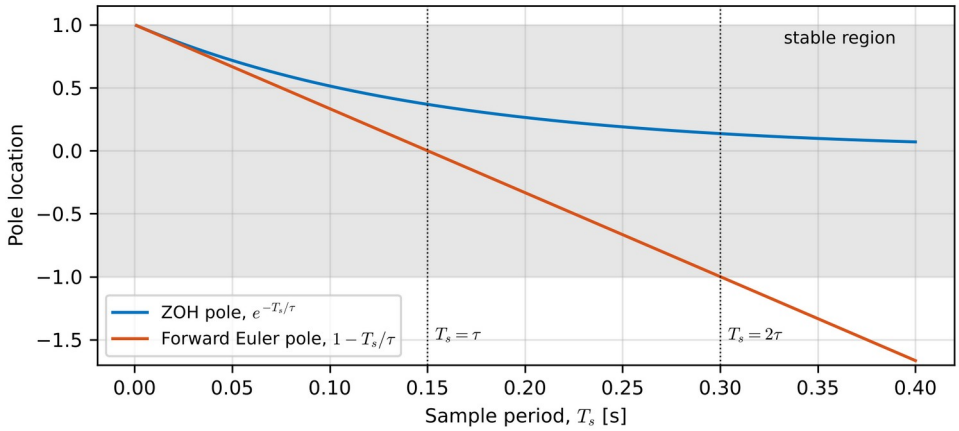


Figure 1: *Theoretical Pole Location of the ZOH and Forward Euler Models against Sample Period, for τ = 0.15 s*

1. Derive both difference equations by hand.
2. Check the ZOH model with `c2d(tf(K,[tau 1]), Ts, 'zoh')` in MATLAB for one sample period.

Run the Models alongside the Motor

[Screenshot to insert] *Signal source (sequence of voltage steps) → Gain 255/5 → PWM output, and the same voltage → MATLAB Function (ZOH model) and MATLAB Function (Euler model); measured speed and both model outputs logged as y, y_zoh, y_eul.*

Figure 2: *Simulink Model Running the Two Discrete-Time Models alongside the Motor*

MATLAB Function Block: one-step update of both discrete-time models

```
function [y_zoh, y_eul] = fcn(u, K, tau, Ts)
persistent yz ye u_prev
if isempty(yz), yz = 0; ye = 0; u_prev = 0; end
a = exp(-Ts/tau);
yz = a*yz + K*(1 - a)*u_prev;         % zero-order-hold model
ye = (1 - Ts/tau)*ye + (K*Ts/tau)*u_prev;   % forward Euler model
u_prev = u;
y_zoh = yz;  y_eul = ye;
```

1. Build the model in Figure 2. Use a Repeating Sequence Stair or similar block to apply a sequence of voltage steps between 1.5 V and 4.5 V, each lasting about 1 s, starting from 1.5 V.
2. Set the initial value of both models to the measured speed at 1.5 V, or let the first second of the test settle before comparing.
3. Run the test at three sample periods: about $\tau/20$, $\tau/5$ and $\tau/2$ (0.0075 s, 0.03 s and 0.075 s for the example motor).
4. For each run, calculate the root-mean-square (RMS) error between each model and the measured speed, and record it in Table 2.

MATLAB Script: RMS error of each model against the measured speed

```
e_zoh = rms(y.signals.values - y_zoh.signals.values)   % [RPM]
e_eul = rms(y.signals.values - y_eul.signals.values)   % [RPM]
```

Table 2: *Model Errors for Your Motor*

T_s (s)	T_s/τ	ZOH pole	Euler pole	ZOH RMS error (RPM)	Euler RMS error (RPM)
0.0075	0.05	0.951	0.950		
0.03	0.20	0.819	0.800		
0.075	0.50	0.607	0.500		

The poles in Table 2 use the example $\tau = 0.15$ s. Recalculate them for your own motor. Both models should track the motor closely at the shortest sample period. At the longest, the ZOH model remains accurate at the sample instants, while the forward Euler model drifts further from the measurement.

1. Explain why the ZOH model is exact at the sample instants when the input is piecewise constant.
2. At what sample period does the forward Euler model become unstable for your motor?
3. What sample period would you choose for a controller on this motor, and why?

4.0 Concluding Remarks

This practical exercise converted a continuous-time motor model into two discrete-time models and compared them with the real motor. Using MATLAB, Simulink and ACE-Lab hardware, the ZOH and forward Euler difference equations were run on the Arduino Uno alongside the motor at several sample periods.

The ZOH model matched the motor at every sample period because the PWM input is held constant between samples. The forward Euler approximation was accurate only when the sample period was small compared with the time constant, and it becomes unstable beyond twice the time constant. Choosing the sample period and the method together is therefore part of designing any digital controller.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Discrete-Time Modelling of a DC Motor. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/23725328